Learning Journal

Ian Silber

June 25, 2026

Contents

---

1 2026-06-25 *Catching up: What I have learned during my first month*

This month has been a fairly eventful one. I have spent most of my time working through Engineering Design via Surrogate Modelling A Practical Guide, as well as reading a few papers discussing the physics my research group is working to model.
What I have worked through thus far:

- Sampling plans, specifically Random Latin Hypercubes and criteria for determining which RLHC's are best.
- Understanding surrogate modelling
- Radial Basis Functions; both fixed and parametric RBFs
- Kriging Method for surrogate modelling, a special form of RBF.

1.1 Sampling Plans: Full Factorial, Random Latin Hypercubes, and Space Filling RLHCs

1.1.1 Full Factorial

```
def full_factorial(q: list) -> None:

    if min(q) < 2:
        raise Exception("You must have at least two points per
            dimension")

    x1 = np.linspace(0,1,q[0])
    x2 = np.linspace(0,1,q[1])
    x3 = np.linspace(0,1,q[2])

```

```
    X1, X2, X3 = np.meshgrid(x1, x2, x3, indexing='ij')

    fig = plt.figure(figsize=(12, 12))

    elevations = [None, 90, 0, 0]
    azimuths   = [70, -90, -90, 0]

    for i, elev, azim in zip(range(1,5), elevations, azimuths):
        ax = fig.add_subplot(2,2,i, projection='3d')

        ax.xaxis.set_major_locator(MultipleLocator(1/(q[0]-1)))
        ax.xaxis.set_minor_locator(MultipleLocator(1/(q[0]-1)))

        ax.yaxis.set_major_locator(MultipleLocator(1/(q[1]-1)))
        ax.yaxis.set_minor_locator(MultipleLocator(1/(q[1]-1)))

        ax.zaxis.set_major_locator(MultipleLocator(1/(q[2]-1)))
        ax.zaxis.set_minor_locator(MultipleLocator(1/(q[2]-1)))

        ax.set_xlabel('x1')
        ax.set_ylabel('x2')
        ax.set_zlabel('x3')
        ax.scatter(X1, X2, X3, alpha=0.5)

        ax.view_init(elev=elev, azim=azim)
        ax.set_box_aspect(None, zoom=0.85)

    plt.show()

full_factorial([3,4,5])
```

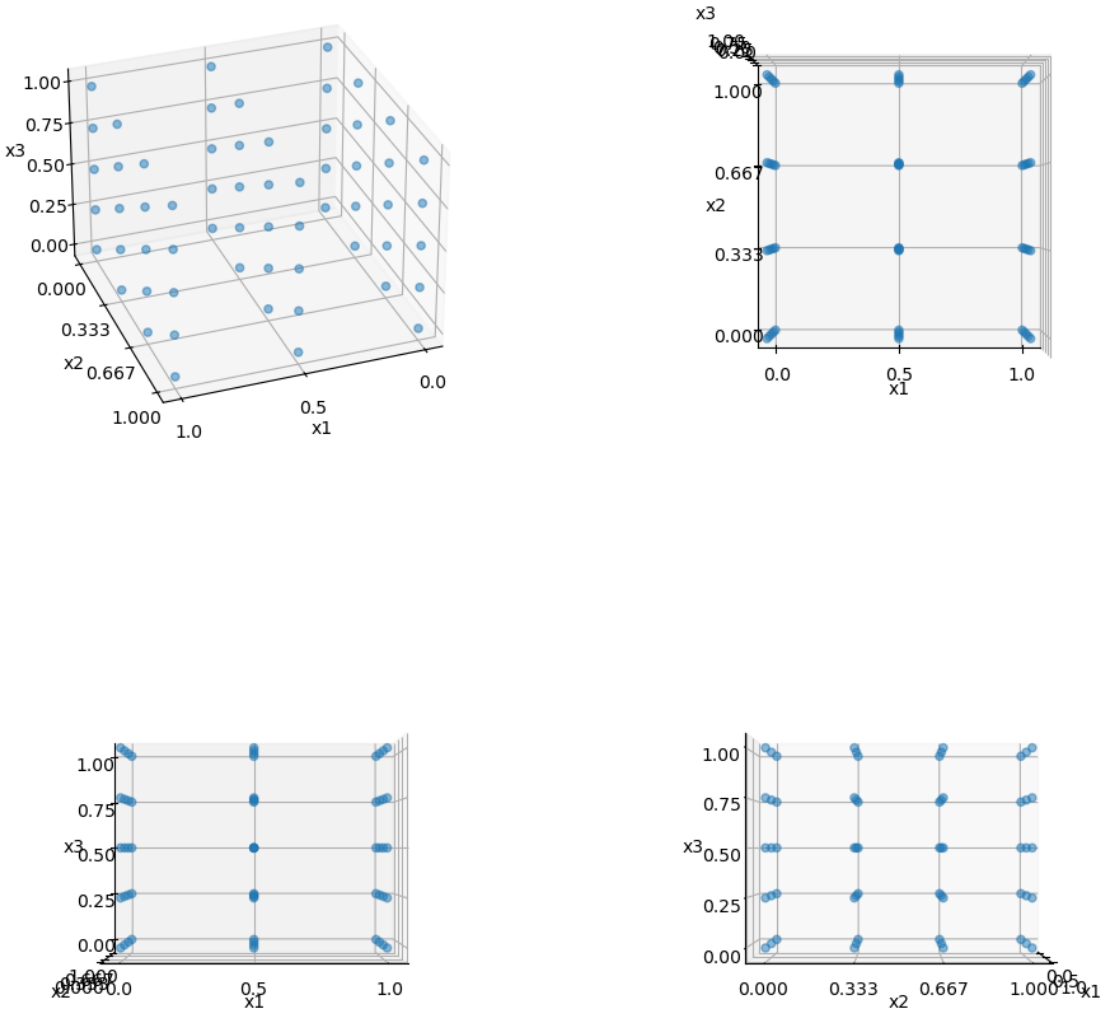

Figure 1: Output of the `space_filling_latin_hypercube` function viewed along different axis.

1.1.2 Random Latin Hypercubes

```python
import numpy as np

def random_latin_hypercube(
        n:int,
        k:int,
        lbs: list = [],
        ubs: list = []
    ) -> np.ndarray:

    rng = np.random.default_rng()

    X = np.zeros((n,k))

    for i in range(k):
        X[:,i] = rng.permutation(n)
```

```
    for i, (ub, lb) in enumerate(zip(ubs, lbs)):
        X[:,i] = (X[:,i] + 0.5)/n
        w = ub - lb

        X_scaled = lb + w*X

    return X_scaled
```

1.1.3 Space Filling Random Latin Hypercubes

Although the above code ensures projection onto the axes does not result in multiple points overlapping, it does not result in maximal space-filling. Hypothetically, this code could produce a sampling plan in which all the points fall along a perfect diagonal through the feature space. This would avoid any overlapping points, but would be far from the most efficient sampling plan. This is where Johnson maximin metric and Morris and Mitchel's "tie-breaker" strategies come together to produce the space filling latin hypercube method.

Definition 1 (Maximin)**.** We call X a maximin plan among all plans if it maximizes d_1 and, among all plans for which this is true minimizes j_1 (where d_1 is distance between possible pairs of points sorted in ascending order, and j_1 is the number of pairs of points in X separated by d_1)

We want to maximize the smallest distance between pairs, and minimize the number of pairs that share this smallest distance.

This will sometimes result in a tie, so Morris and Mitchell's tie breaker method comes into play. Basically, its the same as Maximin but instead of stopping at maximizing d_1 and minimizing j_1, it maximizes $d_1 \dots d_m$ and minimizes $j_1 \dots j_m$.

$$d_p(x^{(i_1)}, x^{(i_2)}) = (\sum_{j=1}^{k} |x_j^{(i_1)} - x_j^{(i_2)}|)^{\frac{1}{p}} \tag{1}$$

The implementation of this plan is predictably a bit more involved:

```
import numpy as np
import math
from scipy.spatial.distance import pdist


def dist(X: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    # # My original approach:
    # distances = []
    # for i in range(len(X)-1):
    #     for j in range(1,len(X)):
    #         distances.append(np.linalg.norm(X[i,:] - X[j,:]))

    # # round because normalized to [0,1]
    # return np.unique(distances, return_counts=True) # returns
       distinct_d, J

    # claude suggested using pdists from scipy instead:
    sq_dists = pdist(X, metric='sqeuclidean')
    return np.unique(sq_dists, return_counts=True) # returns distinct_d
       , J


```

```
def mm_phiq(X,q=2):
    d,J = dist(X)

    # morris mitchell sampling plan quality criterion
    return sum(((J/(d**q))**(1/q)))


def perturb(X: np.ndarray, pert_n=1):
    n = X.shape[0]
    k = X.shape[1]

    for _ in range(pert_n):
        col = int(np.floor(np.random.random()*k))

        el1 = el2 = 1
        while el1==el2:
            el1 = int(np.floor(np.random.random()*n))
            el2 = int(np.floor(np.random.random()*n))

        buffer = X[el1,col]
        X[el1,col] = X[el2,col]
        X[el2,col] = buffer

    return X


def best_lhc_by_q(X_start, pop, iter, q):
    X_best = X_start;
    phi_best = mm_phiq(X_start, q)

    n = X_best.shape[0]

    leveloff = math.floor(0.85*iter)
    for it in range(1,iter+1):
        if it < leveloff:
            mutations = round(1+(0.5*n-1)*(leveloff-it)/(leveloff-1))
        else:
            mutations = 1

        X_improved   = X_best
        phi_improved = phi_best

        for _ in range(pop):
            X_try = perturb(X_best, mutations)
            phi_try = mm_phiq(X_try, q)

            if phi_try < phi_improved:
                X_improved = X_try
                phi_improved = phi_try

        if phi_improved < phi_best:
            X_best = X_improved
            phi_best = phi_improved

    return X_best, phi_best


def space_filling_latin_hypercube(
```

```
        n:int,
        k:int,
        lbs: list = [],
        ubs: list = [],
        pop:int = 25,
        iter:int = 25
    ) -> np.ndarray:
    """
    Generate a random Latin hypercube sampling plan in k dimensions.

    Each dimension is partitioned into n equal-width bins, and the plan
    places exactly one point per bin in every dimension (the Latin
    hypercube property), giving more uniform coverage of the domain
       than
    independent uniform sampling. Points are positioned at bin centres
       and
    then scaled from the unit hypercube [0, 1]^k onto [lb, ub]^k.

    Parameters
    ----------
    n : int
        Number of sample points. Also the number of bins per dimension,
        since the plan places one point per bin.
    k : int
        Number of dimensions (design variables).
    lbs : list, optional
        List of lower bound of the domain in every dimension (default
           0).
    ubs : list, optional
        List of upper bound of the domain in every dimension (default
           1).
        Must satisfy ub > lb.
    pop : int, optional
    iter : int, optional

    Returns
    -------
    np.ndarray
        An (n, k) array of sample points. Each column is a permutation
           of
        the same n bin centres, mapped to [lb, ub].

    Raises
    ------
    ValueError
        if ubs[i] <= lbs[i]

        if not len(ubs) == len(lbs) == k
    """
    X_start = random_latin_hypercube(n,k,lbs=lbs,ubs=ubs) # get
       sampling plan

    qs = [1,2,5,10,20,50,100]

    X_tracker = {
        "q": "",
        "phi": math.inf,
        "X": ""
```

```
    }

    for q in qs:
        X_best, phi_best = best_lhc_by_q(X_start, pop, iter, q)

        if phi_best < X_tracker["phi"]:
            X_tracker["q"]   = q
            X_tracker["phi"] = phi_best
            X_tracker["X"]   = X_best

    return X_best


space_filling_latin_hypercube(25,3,25,25)
```

Which generates the below sampling plan:

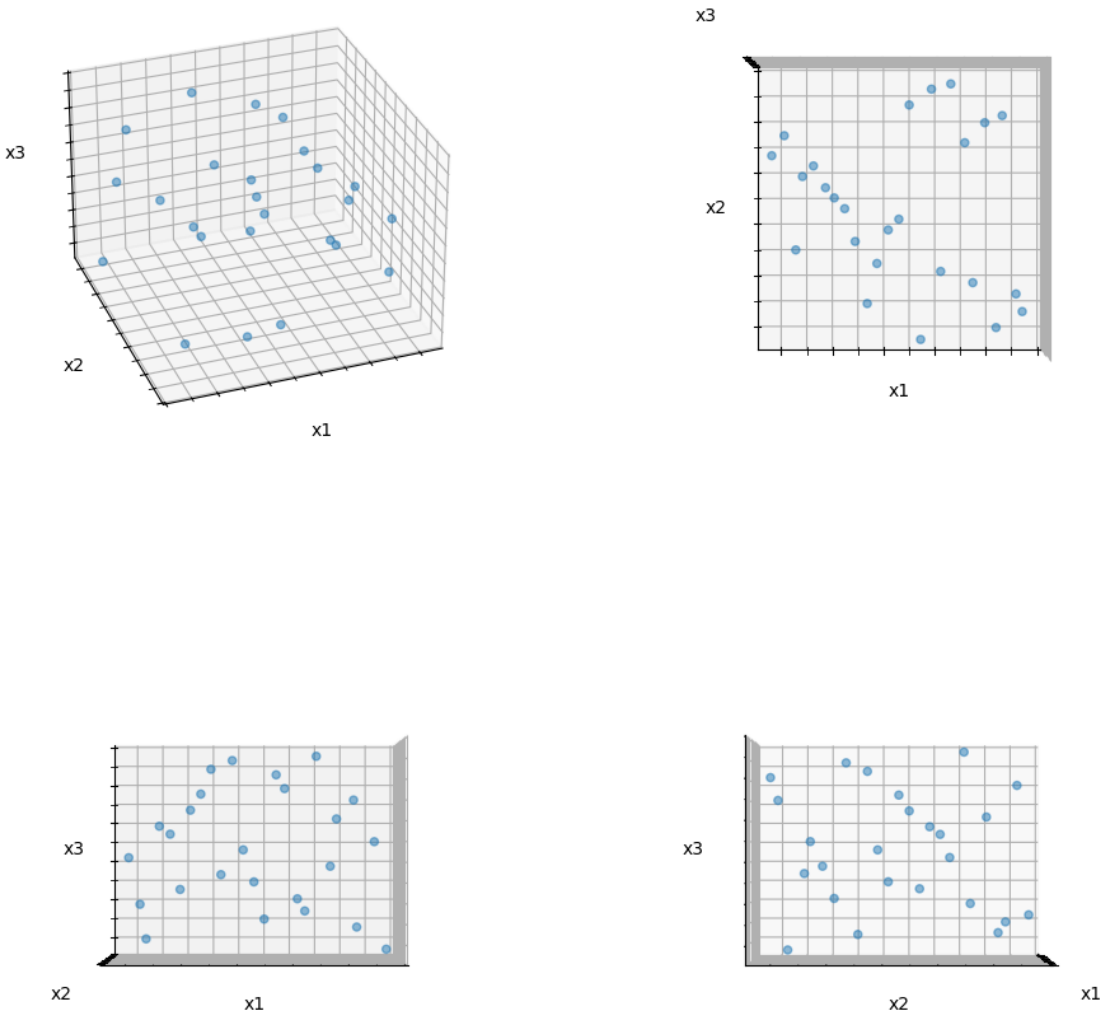

Figure 2: Output of the `space_filling_latin_hypercube` function viewed along different axis.

1.2 Radial Basis Functions: fixed and parametric

This should be below the figure

1.3 Kriging: A special form of RBF